

Sea++ Programming Language Specification v1.0

Introduction

Sea++ is a high-level, object-oriented language similar to C++, but using the keywords `begin` and `end` instead of `{ }` to delimit blocks.

Chapter 1: Tokens (Phase 1 – Lexer)

1.1 Keywords

`begin`, `bool`, `break`, `case`, `char`, `class`, `do`, `double`, `else`, `end`, `false`, `float`, `for`, `if`, `int`, `return`, `string`, `switch`, `true`, `try`, `void`, `while`

All keywords are lowercase.

1.2 Operators

- **Arithmetic:** `+`, `-`, `*`, `/`, `%`
- **Increment/Decrement:** `++`, `--`
- **Relational:** `<`, `>`, `<=`, `>=`, `==`, `!=`
- **Logical:** `&&`, `||`, `!`
- **Assignment:** `=`
- **Member access:** `.`

1.3 Delimiters

`;`, `(`, `)`, `{`, `}`, `[`, `]`, `::`, `.`

1.4 Literals

- **Integer Literal:** `[0-9]+` (Examples: `0`, `42`, `100`)
- **Single-precision Float Literal:** `[0-9]+\.[0-9]+f` (Examples: `3.14f`, `0.0f`, `10.5f`)
- **Double-precision Float Literal:** `[0-9]+\.[0-9]+` (Examples: `3.14`, `0.0`, `10.5`)
- **String Literal:** `"[^"]*"` (Examples: `"Hello world!"`)

1.5 Identifiers

Pattern: `[a-zA-Z_][a-zA-Z0-9_]*`. Must start with a letter or underscore. Case-sensitive; cannot be a keyword.

1.6 Comments

- **Single-line:** `// ...`
- **Multi-line:** `/* ... */` (Does not nest).

1.7 Whitespace

Spaces, tabs (`\t`), and newlines (`\n`) are ignored by the lexer except for line and column counting.

Chapter 2: Grammar (Phase 2 – Parser)

2.1 Notation

Terminals are in quotes; non-terminals are *italicized*. `?` denotes zero or one, `*` zero or more, `+` one or more, and `|` indicates alternatives.

2.2 Main Grammar

- $program \rightarrow (class_definition \mid function_definition)^*$
- $class_definition \rightarrow "class" \text{ IDENTIFIER } "begin" (variable_declaration \mid function_definition)^* "end"$
- $function_definition \rightarrow type \text{ IDENTIFIER } "(" parameter_list? ")" function_body$
- $function_body \rightarrow "begin" statement^* "end" \mid statement$
- $parameter_list \rightarrow parameter ("," parameter)^*$
- $parameter \rightarrow type \text{ IDENTIFIER }$
- $type \rightarrow "int" \mid "float" \mid "double" \mid "string" \mid "bool" \mid "char" \mid "void" \mid \text{IDENTIFIER}$

2.3 Statements

- $statement \rightarrow variable_declaration ";" \mid assignment_statement ";" \mid if_statement \mid while_statement \mid for_statement \mid return_statement ";" \mid function_call ";" \mid ";"$
- $variable_declaration \rightarrow type \text{ IDENTIFIER } ("=" expression)?$
- $assignment_statement \rightarrow \text{IDENTIFIER } ("." \text{ IDENTIFIER })? "=" expression$
- $if_statement \rightarrow "if" "(" expression ")" if_body ("else if" "(" expression ")" if_body)^* ("else" if_body)?$
- $if_body \rightarrow "begin" statement^* "end" \mid statement$

- *while_statement* → "while" "(" *expression* ")" *while_body*
- *while_body* → "begin" *statement** "end" | *statement*
- *for_statement* → "for" "(" *for_init* ";" *expression* ";" *for_update* ")" *for_body*
- *for_init* → *variable_declaration* | *assignment_statement*
- *for_update* → *assignment_statement*
- *for_body* → "begin" *statement** "end" | *statement*
- *return_statement* → "return" *expression*?
- *function_call* → IDENTIFIER "(" *argument_list*? ")"
- *argument_list* → *expression* ("," *expression*)*

2.4 Expressions

- *expression* → *logical_or* ("?" *expression* ":" *expression*)?
- *logical_or* → *logical_and* ("|" *logical_and*)*
- *logical_and* → *equality* ("&&" *equality*)*
- *equality* → *relational* ("==" *relational* | "!=" *relational*)*
- *relational* → *additive* ("<" *additive** | ">" *additive* | "<=" *additive** | ">=" *additive*)
- *additive* → *multiplicative* ("+" *multiplicative* | "-" *multiplicative*)*
- *multiplicative* → *unary* (" " *unary* | "/" *unary* | "%" *unary*)
- *unary* → ("+" | "-" | "++" | "--" | "!")? *primary*
- *primary* → INTEGER_LITERAL | FLOAT_LITERAL | DOUBLE_LITERAL | STRING_LITERAL | "true" | "false" | IDENTIFIER | IDENTIFIER "." IDENTIFIER | "(" *expression* ")"

Chapter 3: Additional Syntactic & Semantic Rules

3.1 The main Function

Every program must have exactly one main() function. Return type: void or int.

3.2 Block (Scope) Rule

If a body contains exactly one statement, begin/end may be omitted. If it contains two or more, they are required.

3.3 Data Types

- **int:** 32-bit signed integer
 - **float:** 32-bit IEEE-754 single precision
 - **double:** 64-bit IEEE-754 double precision
 - **string:** sequence of characters
 - **bool:** true or false
 - **char:** single ASCII character
 - **void:** no value
-

Chapter 4: Parser Output Requirements

- **Class:** Class: <class_name>
 - **Function:** Function: <return_type> <name>(<params>)
 - **Variable:** Variable: <type> <name> (= <value>)?
 - **Conditional/Loop:** Conditional: <type> or Loop: <type>
 - **Call:** Call: <name>(<args>)
 - **Math:** Math Expression Result: <value>
-

Chapter 5: Bonus Features (Semantic Analyzer)

1. **AST Construction:** Generate an Abstract Syntax Tree.
 2. **Uniqueness:** Class, function, and variable names must be unique within their scope.
 3. **Cyclic Dependencies:** Print Warning: Cyclic dependency detected between class A and class B.
-